Lakehouse Performance: Best Practices & Production Readiness

A field guide for platform and data teams · files, layout, maintenance, masks, queries, warehouses · Databricks · October 2026

HOW TO USE THIS GUIDE

Each section gives the **rule**, **why it matters**, how to **check** it and how to **fix** it. Every command was run end-to-end in a Databricks workspace before publishing, and every product rule links to the Databricks documentation it comes from. Three companion **SQL notebooks** (run them on a SQL warehouse) automate the checks. They're in github.com/althrussell/interesting-things → performance-diagnostics:

- **Production_Readiness_Check**: PASS / WARN / FAIL for a data product across every layer below (section 10).
- **Performance_Alerts**: five daily early-warning SQL alerts for the platform team (section 9).
- **Column_Masking_Performance_Diagnostic**: finds the tables, masks and queries costing the most time, split into planning vs execution.

The short version: 12 rules

#	Rule	Section
1	Use **Unity Catalog managed tables** and keep **Predictive Optimization** on, so OPTIMIZE, VACUUM and ANALYZE run for you.	3
2	Use **liquid clustering** (or `CLUSTER BY AUTO`) for every new table. Don't partition tables under 1 TB.	2
3	Don't use **bloom filter indexes** or **Z-ORDER** on new tables.	2
4	Keep data files near their autotuned size (256 MB for most tables). Fix small files with `OPTIMIZE`.	1
5	Keep **VACUUM retention at 7 days or more**.	3
6	**Never read a table inside a column mask or row filter.** Decide access with `is_account_group_member()` / `current_user()`.	4
7	Mask at scale with **ABAC policies on governed tags**. Keep mask functions simple, in SQL, and `DETERMINISTIC`.	4
8	Name columns in BI and production queries. Avoid `SELECT *` on wide or masked tables.	5
9	Read the **query profile**: planning time and execution time have different fixes.	5
10	**Spill → bigger warehouse. Queueing → more clusters.** Prefer serverless. Keep 2X-Small for development.	6
11	Run a **daily alert** on queueing, spill, slow queries, planning time and failures, so you hear before users do.	9
12	Run a **production-readiness check** before any new data product goes live, and again 2–4 weeks after.	10

1 · Data files (Parquet file sizes)

Rule. Let Databricks size files, and compact when files get small. **Why:** every file costs a metadata lookup and a read request, so thousands of tiny files make both planning and scans slow.

- Databricks autotunes target file size by table size: *"For tables smaller than 2.56 TB, the autotuned target file size is 256 MB. For tables with a size between 2.56 TB and 10 TB, the target size grows linearly from 256 MB to 1 GB. For tables larger than 10 TB, the target file size is 1 GB."* Leave `delta.targetFileSize` unset unless you have a measured reason.
- *"Auto compaction and optimized writes each reduce small file problems, but are not a full replacement for `OPTIMIZE`."* Predictive Optimization runs `OPTIMIZE` for you on managed tables.
- Rule of thumb used by our readiness check: an average file well under ~64 MB across 50+ files means compaction is overdue.

Check, then fix

```
-- numFiles and sizeInBytes → average file size
DESCRIBE DETAIL main.sales.orders;

-- compact (and apply clustering, if the table is clustered)
OPTIMIZE main.sales.orders;
```

Docs: Configure Delta Lake to control data file size

2 · Layout: liquid clustering, partitions, Z-order, bloom filters

Rule. *"Databricks recommends liquid clustering for all new tables, including streaming tables and materialized views."* Pick **up to four** columns you filter or join on, or let Databricks choose with `CLUSTER BY AUTO`. That requires Predictive Optimization and Unity Catalog managed tables.

Partitioning	*"With less than 1 TB of data, don't partition."* From 1 to 100 TB, *"use liquid clustering instead of partitioning"*. If you do partition, *"verify that each partition contains at least 1 GB of data"*, and only on low-cardinality columns such as dates.
Z-ORDER	*"Clustering is not compatible with partitioning or `ZORDER`."* Use liquid clustering instead.
Bloom filters	*"Do not use Bloom filter indexes. Databricks has deprecated this feature and recommends removing any existing Bloom filter indexes from your tables."* Predictive I/O and liquid clustering replace them.
Wide external tables	For external tables, skipping statistics cover *"the first 32 columns defined in your table schema by default."* Set `delta.dataSkippingStatsColumns` to your filter columns. Managed tables with Predictive Optimization pick the most-filtered columns automatically.

New table

```
CREATE TABLE main.sales.orders (
  order_date DATE, customer_id BIGINT, region STRING, amount DOUBLE)
CLUSTER BY (order_date, customer_id);      -- or: CLUSTER BY AUTO
```

Existing unpartitioned table

```
-- or CLUSTER BY AUTO
ALTER TABLE main.sales.orders CLUSTER BY (order_date, customer_id);
-- clusters the existing data
OPTIMIZE main.sales.orders;
```

Existing partitioned table: convert in place

```
-- or AUTO
ALTER TABLE main.sales.orders_legacy
  REPLACE PARTITIONED BY WITH CLUSTER BY (order_date);
OPTIMIZE main.sales.orders_legacy;
```

Remove legacy bloom filters; set stats columns on a wide external table

```
DROP BLOOMFILTER INDEX ON TABLE main.sales.orders_legacy
  FOR COLUMNS (customer_ref);
ALTER TABLE main.ext.wide_feed SET TBLPROPERTIES (
  'delta.dataSkippingStatsColumns' = 'order_date,customer_id');
```

Docs: Use liquid clustering · When to partition · Bloom filter indexes · Data skipping

3 · Delta maintenance: OPTIMIZE, VACUUM, ANALYZE

Rule. Let **Predictive Optimization** do it: *"Predictive optimization automatically runs `OPTIMIZE`, `VACUUM`, and `ANALYZE` on Unity Catalog managed tables."* It's *"enabled by default for accounts created on or after November 11, 2024"*, and you can enable it per catalog or schema. External tables aren't covered, so schedule maintenance for those yourself.

Check it's on, and that it's actually running

```
-- "Predictive Optimization: ENABLE / DISABLE / INHERIT"
DESCRIBE CATALOG EXTENDED main;
DESCRIBE SCHEMA EXTENDED main.sales;

SELECT operation_type, count(*) AS operations,
       count(DISTINCT table_id) AS tables, date(max(end_time)) AS latest
FROM system.storage.predictive_optimization_operations_history
WHERE end_time > current_timestamp() - INTERVAL 30 DAYS
  AND operation_status = 'SUCCESSFUL'
GROUP BY operation_type ORDER BY operations DESC;
```

Turn it on

```
ALTER CATALOG main ENABLE PREDICTIVE OPTIMIZATION;
-- or INHERIT to follow the catalog
ALTER SCHEMA main.sales ENABLE PREDICTIVE OPTIMIZATION;
```

When you must run it yourself (external tables, or no Predictive Optimization)

```
-- daily is the documented guidance
OPTIMIZE main.sales.orders;
-- feeds the cost-based optimizer
ANALYZE TABLE main.sales.orders COMPUTE STATISTICS FOR ALL COLUMNS;
-- default 7-day retention
VACUUM main.sales.orders;
-- faster; uses the transaction log
VACUUM main.sales.orders LITE;
```

VACUUM retention. *"Databricks strongly recommends setting a retention interval of at least 7 days."* Shorter retention can delete files that long-running jobs haven't committed yet, and it removes time travel beyond the window. To reset it:

```
ALTER TABLE main.sales.orders SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration' = 'interval 7 days');
```

Docs: Predictive optimization · VACUUM · ANALYZE TABLE · Cost-based optimizer

4 · Column masks and row filters

A mask costs time in two different ways:

Cost	When it's paid	What drives it	What helps
Execution	Per row, only for masked columns the query selects	Mask language: SQL is cheap; Python is ~17× slower in our tests and can't be a mask on serverless	SQL masks; explicit column lists
Planning	On *every* query on the table, for *every* mask on it, whatever columns are selected	Masks that **read a table** (for example a permission lookup)	Removing the table lookup: ABAC policies, or masks that only check groups

MEASURED: TABLE LOOKUPS INSIDE MASKS

A mask that ran four permission lookups against two governance tables, on a 208-column table with 52 such masks, selecting 10 *unmasked* columns with `LIMIT 5`. Same access rules in all three columns:

Measured	Lookup masks	Masks that check groups only	ABAC policies
Serverless notebook / job: planning	**34–39 s**	~1.5 s	~1.1 s
SQL warehouse: whole query, client round trip	**5.6–9.9 s**	1.6–2.0 s	~1.6 s

Planning grew linearly, about **0.65 s per lookup mask on serverless and 0.16 s on a SQL warehouse** (13 → 9 s, 26 → 17 s, 52 → 34 s on serverless). On the warehouse, ≈4 s of planning remained even when the result came from cache. Narrow views, column lists, `LIMIT` and being in a bypass group **did not reduce it**.

Recommended: ABAC policies on governed tags

"Databricks recommends ABAC policies when you need consistent row filtering and column masking across many tables." A policy is attached once to a catalog or schema. It decides *who* is masked from group membership (`TO` / `EXCEPT`), *which columns* from their governed tags, and applies to every matching table, including new ones as soon as their columns are tagged. The mask function only transforms the value.

```
-- 1. A governed tag (account level). Or in the UI:
--    Catalog → Govern → Governed Tags → Create governed tag
CREATE GOVERNED TAG pii_mask
  DESCRIPTION 'PII column; the value is the mask type to apply'
  VALUES ('text', 'number');

-- 2. One simple mask function per data type
CREATE OR REPLACE FUNCTION main.governance.mask_text(v STRING)
RETURNS STRING DETERMINISTIC
RETURN '***';

CREATE OR REPLACE FUNCTION main.governance.mask_number(v DECIMAL(38, 10))
RETURNS DECIMAL(38, 10) DETERMINISTIC
RETURN NULL;
```

```
-- 3. One policy per data type, for every table in the schema
CREATE OR REPLACE POLICY mask_pii_text
ON SCHEMA main.sales
COLUMN MASK main.governance.mask_text
TO `account users` EXCEPT `pii_readers`
FOR TABLES
MATCH COLUMNS has_tag_value('pii_mask', 'text') AS c
ON COLUMN c;

CREATE OR REPLACE POLICY mask_pii_number
ON SCHEMA main.sales
COLUMN MASK main.governance.mask_number
TO `account users` EXCEPT `pii_readers`
FOR TABLES
MATCH COLUMNS has_tag_value('pii_mask', 'number') AS c
ON COLUMN c;
```

```
-- 4. Tag columns: the matching policy applies straight away
ALTER TABLE main.sales.customers
  ALTER COLUMN credit_limit SET TAGS ('pii_mask' = 'number');

-- See what's in force
SHOW POLICIES ON SCHEMA main.sales;
DESCRIBE POLICY mask_pii_text ON SCHEMA main.sales;
```

Rules that keep ABAC fast and safe

- **Scope each policy to one data type** with a tag value, as above. A policy whose function takes a STRING fails on a DECIMAL column it matches. One `DECIMAL(38, 10)` function covers every decimal column.

- **One mask per column per user.** *"Only one distinct column mask can resolve for a given column and a given user."* If two different masks apply, Databricks blocks the query with an error. That includes a table-level mask plus a policy.
- **Several access levels?** Add a second tag (for example `access_set = 'finance'`) and combine conditions: `has_tag_value('pii_mask', 'text') AND NOT has_tag('access_set')` for the general policy, and a second policy for `has_tag_value('access_set', 'finance')` with the finance group in `EXCEPT`. Up to 3 conditions per `MATCH COLUMNS`.
- **Quotas:** 100 policies per catalog and per schema, 50 per table, 20 principals per policy (use groups, not users).
- **Compute:** serverless, or Databricks Runtime 16.4+ on standard or dedicated compute. Older runtimes can't read ABAC-secured tables.
- **Materialized views and streaming tables** refresh as the pipeline owner. Put the owner in `EXCEPT`, or the stored data is masked for everyone.
- Policy masks don't appear in `information_schema.column_masks`; use `SHOW POLICIES`. Deleting a governed tag that a policy uses makes queries in the policy's scope fail. If `CREATE POLICY` reports *Unknown tag policy key* just after you create a tag, wait a minute and retry.

Moving a column from a table mask to a policy: tag first, then drop

Only drop the old mask once a policy matches the column's tags: a tagged column that no policy matches is left unmasked. The coverage check below confirms it. Move one column at a time.

```
-- Between these two statements, queries on the column fail (two
-- masks) rather than return unmasked data.
ALTER TABLE main.sales.customers
  ALTER COLUMN email SET TAGS ('pii_mask' = 'text');
ALTER TABLE main.sales.customers ALTER COLUMN email DROP MASK;
```

Running ABAC day to day

Groups carry the day-to-day change: grant access by group membership, managed in your identity provider, and never name individual users in policies. Keep policies in version control with your deployment code; there are few of them and they rarely change. Set column tags in the same

deployment step that creates the table, and grant `ASSIGN` on governed tags only to that deployment and your data stewards. Everything else is queryable:

```
-- Every policy and who is exempt
SELECT policy_name, except_principals, match_columns
FROM system.information_schema.abac_policy_definitions
WHERE catalog_name = 'main' AND schema_name = 'sales';

-- Coverage check: columns tagged for masking that no policy matches,
-- so NOT masked. Should return no rows; run it daily as a SQL alert.
WITH covered AS (
  SELECT DISTINCT g.tag_value
  FROM system.information_schema.abac_policy_definitions
  LATERAL VIEW explode(governed_tags) t AS g
  WHERE catalog_name = 'main' AND schema_name = 'sales'
    AND g.tag_key = 'pii_mask')
SELECT table_name, column_name, tag_value
FROM system.information_schema.column_tags
WHERE catalog_name = 'main' AND schema_name = 'sales'
  AND tag_name = 'pii_mask'
  AND tag_value NOT IN (SELECT tag_value FROM covered);

-- Who changed a policy or a column tag, and when
SELECT event_time, user_identity.email AS changed_by, action_name
FROM system.access.audit
WHERE event_date >= current_date() - INTERVAL 30 DAYS
  AND service_name = 'unityCatalog'
  AND action_name IN ('createPolicy', 'updatePolicy', 'deletePolicy',
                      'UpdateTagSubentityAssignments')
ORDER BY event_time DESC;
```

Writing any mask or filter function

From the Databricks UDF guidance for policies: *"Keep UDFs simple. Favor basic `CASE` statements and simple boolean expressions." "Avoid multi-level nesting and unnecessary function calls." "Avoid External API calls or lookups to other databases in UDFs." "Use deterministic expressions that cannot throw errors in policy UDFs." "Use SQL UDFs instead of Python UDFs whenever possible."*

If you keep table-level masks

For a few tables with table-specific logic, decide access with `is_account_group_member('group')`, which *"returns true if the session (connected) user is a direct or indirect member of the specified group at the account level"*, or `current_user()` for named users. If your rules live in a configuration table, **move them to ABAC** rather than reading the table inside a mask: its rows become group memberships (who), column tags (what) and a few policies (the rule). Don't copy the rules into

generated masks either: every rule change then needs a redeploy, and the masks drift from the table whenever one is missed.

```
CREATE OR REPLACE FUNCTION main.governance.mask_pii_string(v STRING)
RETURNS STRING DETERMINISTIC
RETURN CASE WHEN is_account_group_member('pii_readers') THEN v ELSE '***' END;

ALTER TABLE main.sales.customers
  ALTER COLUMN email SET MASK main.governance.mask_pii_string;
```

Find table masks that read tables (the expensive kind)

```
SELECT cm.table_catalog, cm.table_schema, cm.table_name,
       count(*) AS lookup_masks
FROM system.information_schema.column_masks cm
JOIN system.information_schema.routines r
  ON concat(r.routine_catalog, '.', r.routine_schema, '.', r.routine_name)
     = cm.mask_name
WHERE r.sql_data_access = 'READS_SQL_DATA'
GROUP BY 1, 2, 3 ORDER BY lookup_masks DESC;
```

Docs: ABAC · Create policies · ABAC requirements and quotas · UDF best practices · Governed tags · Row filters and column masks · is_account_group_member

5 · Queries

- **Read the query profile** (Query History → the query → *See query profile*). It shows *"each query operator and related metrics, such as the time spent, number of rows processed… and memory consumption."* Long **compilation** with a short run points to masks or very wide/nested views; long execution points to spill, shuffle or poor pruning.
- **Name columns** in BI models and production SQL. `SELECT *` on a wide masked table makes every mask execute.
- **Result cache.** SQL warehouses cache results for 24 hours, and the cache is *"invalidated when the underlying tables are updated."* Repeated dashboard queries are cheap when nothing changed. It doesn't skip mask planning.
- **Joins and aggregations**: filter early; join on clustered columns; avoid exploding joins. Multi-GB shuffle or spill in the profile is the signal.

Planning-bound queries: what execution-time rankings miss

```
SELECT round(compilation_duration_ms / 1000.0, 1) AS planning_s,
       round(execution_duration_ms / 1000.0, 1)   AS execution_s,
       executed_by, left(statement_text, 100) AS statement_preview,
       statement_id
FROM system.query.history
WHERE start_time > current_timestamp() - INTERVAL 7 DAYS
  AND compilation_duration_ms > 5000
  AND compilation_duration_ms > execution_duration_ms
ORDER BY compilation_duration_ms DESC LIMIT 50;
```

Docs: Query profile · Query history system table · Query caching

6 · SQL warehouses

Size vs clusters	*"Cluster size… determines the compute resources available for a single cluster. The autoscaler adds or removes clusters of that size as needed."* **Size** buys memory per query; **max clusters** buys concurrency.
Spill	*"If queries are spilling to disk, increase the cluster size. Check for spills in the query profile."*
Queueing	Raise **max clusters**. Classic and pro warehouses *"have a fixed limit of one cluster per 10 concurrent queries."*
Serverless	Prefer serverless SQL warehouses. Intelligent Workload Management starts a query immediately when capacity exists and queues it otherwise.
Production floor	Size production warehouses from a load test; keep 2X-Small for development. Separate BI, ETL and ad-hoc workloads onto different warehouses.

Find the pressure, per warehouse

```
SELECT compute.warehouse_id,
       round(sum(coalesce(spilled_local_bytes, 0)) / power(1024, 3), 1)
         AS spill_gib,
       round(sum(coalesce(waiting_at_capacity_duration_ms, 0)) / 60000.0, 1)
         AS queue_min,
       round(sum(coalesce(waiting_for_compute_duration_ms, 0)) / 60000.0, 1)
         AS cold_start_min,
       round(percentile_approx(total_duration_ms, 0.95) / 1000.0, 1) AS p95_s
FROM system.query.history
WHERE start_time > current_timestamp() - INTERVAL 7 DAYS
  AND compute.warehouse_id IS NOT NULL
GROUP BY 1 ORDER BY spill_gib DESC;
```

The **DBSQL RightSize Advisor** dashboard (same repository) turns this into per-warehouse recommendations.

Docs: SQL warehouse sizing, scaling and queuing

7 · Governance and design patterns that scale

- One access decision per *principal*, not per *row* or per *column lookup*: groups and tags, not tables queried at runtime.
- Mask only what needs masking; review heavily-masked tables (25+ masked columns) for columns that no longer need it.
- Avoid building masked views on top of masked views. Each layer is resolved during planning.
- Tag everything consistently (environment, domain, sensitivity). Use **governed tags** for anything a policy keys on: they're account-wide, only allowed values can be set, and only permitted users can assign them.
- Build every new data product on **catalog- or schema-level ABAC policies** from day one. Tagging a column is then the only step needed to protect it.

8 · Don't do this again

Don't	Do instead	Why
Read a governance or lookup table inside a column mask or row filter	ABAC policies on governed tags, or masks that only check `is_account_group_member()`	~0.16–0.65 s of planning per mask on every query (section 4)
Copy a rules table into generated mask functions	Groups (who), tags (what) and ABAC policies (the rule)	Every rule change needs a redeploy; masks drift from the table
Bind masks table by table across many tables	ABAC policies on the catalog or schema, keyed on governed tags	Hundreds of bindings to keep in step; new tables get missed
Drop a table mask before a policy covers the column	Tag first, run the coverage check, then drop	Avoids a window with unmasked data
Expect narrow views or `LIMIT` to hide mask cost	Fix the mask itself	Planning cost is paid for every mask on the table
Write masks or filters in Python	SQL masks, marked `DETERMINISTIC`	Slower; not supported as masks on serverless
Partition tables under 1 TB, or on high-cardinality columns	Liquid clustering / `CLUSTER BY AUTO`	Small files and skew
Create bloom filter indexes or Z-ORDER new tables	Liquid clustering; drop existing bloom filters	Deprecated / incompatible with clustering
Turn Predictive Optimization off, or rely on people remembering OPTIMIZE	Predictive Optimization on every catalog	Stale statistics, small files, storage growth
Lower VACUUM retention below 7 days	Keep ≥ 7 days	Can break long-running jobs and time travel
`SELECT *` in BI models on wide or masked tables	Explicit column lists	Every mask executes; more I/O
Run production on 2X-Small, or one warehouse for everything	Load-tested size; separate BI / ETL / ad-hoc	Spill and queueing
Rank only by execution time	Also track planning time	Planning-bound queries are invisible otherwise
Find out about slowness from users	Daily early-warning alerts (section 9)	Fix before it's felt
Go live without a readiness check	Readiness gate (section 10)	Catch design issues while they're cheap

9 · Early warning: daily SQL alerts for the platform team

Databricks SQL alerts *"run queries on a schedule and notify you when a condition that you define is met against the query result."* The **Performance_Alerts** notebook holds five alert queries over the **last 24 hours** of `system.query.history`. Schedule each one **once a day**, for example at 07:00, so the

team starts the day with a short list of what to fix. These alerts track trends; they aren't meant to page anyone.

Alert (last 24 hours)	Fires when	First action
Warehouse queueing	a warehouse queued > 30 min of query time	Raise max clusters
Disk spill	a warehouse spilled > 25 GiB	Increase size, or tune the spilling query
Slow queries	> 25 queries over 60 s on a warehouse	Check the other alerts, then query profiles
Planning-time regression	any query planned > 30 s, longer than it ran	Look for table-lookup masks or very wide/ nested views
Failure rate	> 10% of a warehouse's queries failed (50+ queries)	Read `error_message` in query history

The thresholds come from a week of a production estate's history: each alert fired on 1–4 of its 16 warehouses, not on all of them. The notebook's last cell shows your own daily p50 and p90 for each value, so you can tune the thresholds after a week or two.

Set up (about 15 minutes)

1. Import `Performance_Alerts` (Workspace → Import → File), attach a SQL warehouse and run it to see what each alert would report today.
2. For each alert: **Alerts** → **Create alert**, paste the query, pick a small serverless SQL warehouse. Set **Condition** to the value column, **Max**, **>** and the threshold, with **Empty result state** set to OK.
3. **Schedule:** every 1 day at 07:00 in your time zone.
4. **Notify:** the team's email, or a Slack, Teams or webhook notification destination. Set the repeat notification frequency to 1 hour so it re-sends at each daily run while the problem lasts, and turn on **Notify on OK**.

Docs: Databricks SQL alerts · Query history system table

10 · Production readiness for new data products

Most "it's slow in production" problems are design decisions that were cheap to change before go-live: a lookup inside a mask, a partitioned small table, a 2X-Small warehouse. Build three checkpoints into every new data product (for example each new "360"):

When	Checkpoint	What we look at	Exit criteria
Design	Design review	Data model, layout (clustering keys), masking and governance pattern, warehouse plan, expected concurrency	No lookup-in-mask designs; clustering keys chosen; warehouse plan agreed
Before go-live	Readiness gate	**Production_Readiness_Check** on the product's schemas, after a representative test workload; one query profile per key report	No FAIL; every WARN accepted by a named owner; alerts in place
2–4 weeks after	Post-launch check	Re-run the readiness check on real usage; RightSize dashboard; alert history	Same as above, on real load

Monthly	Health check	Readiness check across live products; trend in alerts	Actions logged and owned

Running the readiness check

1. Import `Production_Readiness_Check` (a SQL notebook) and attach a SQL warehouse. Set `target_catalog` and `target_schemas` in the first cell.
2. Run it after a representative test workload, so the query and warehouse checks have something to measure.
3. Read the verdict: **READY**, **READY WITH WARNINGS** or **NOT READY**. Each row carries the evidence and the fix. Optionally set `results_table` to keep a history of runs.
4. Do the guided layout step at the end (a few minutes): it lists a `DESCRIBE DETAIL` for each table and a `DESCRIBE SCHEMA EXTENDED` for each schema, with what healthy output looks like and the fix if it isn't.

Layer	Checks (FAIL in bold)
Masks & filters	**10+ table-lookup masks on a table** (fewer: warn) · **Python masks** · non-deterministic masks · heavily-masked tables · lookup row filters · table-by-table masks where ABAC policies would scale
Maintenance	managed tables with no Predictive Optimization operations in 30 days
Files & layout	partitioned tables · external tables · *guided:* clustering, file size, VACUUM retention, Predictive Optimization setting
Queries	**planning-bound queries** (> 5 s and longer than execution) · slow p95 · `SELECT *` on heavily-masked tables · spill
Warehouses	spill · capacity queueing · cold starts (classic/pro) · 2X-Small · failure rate

Appendix · How this guide was tested

Every SQL statement and notebook above was run end-to-end on Databricks (serverless SQL warehouse and serverless notebook/job compute) in October 2026, with test catalog, schema and group names substituted. The fixtures covered good and bad tables: clustered, partitioned, small-file, wide, 1-day retention, lookup and simple masks, non-deterministic masks and lookup row filters. The readiness check returned the expected PASS / WARN / FAIL for each. The ABAC steps were run as written: governed tag, type-scoped policies, tagging, the two-mask error between "tag" and "drop", and the policy taking over once the table mask was dropped. The five alerts were created on a daily schedule and evaluated by that schedule (the planning alert TRIGGERED on a deliberate 52-lookup-mask test table, four OK), then removed. Some branches can't be exercised on serverless-only test compute and were reviewed in code only: Python masks (unsupported on serverless), external tables, 2X-Small warehouses and high failure rates. Product behaviour is quoted from the Databricks documentation linked in each section. Please re-confirm anything subscription-specific (for example Predictive Optimization or ABAC availability) for your workspace.